Relatorio dos Scripts C# e Spawn

TCC-Game - Assets/Scripts - analise em 16/09/2026

Diagnostico do erro

O erro 'Navigation spawn anchor CenterDefaultSpawn is missing' ocorre porque PlayerSpawnController procura um GameObject ATIVO com esse nome exato na cena ativa.

Nao e um problema do prefab Player, Aventureiro ou Canvas. A busca e GameObject.Find(nomeDaAncora). Sem a ancora, o controlador nao reposiciona o Player.

Para uma nova cena de centro: crie um Empty GameObject chamado CenterDefaultSpawn e posicione-o onde o Player deve nascer. Nao requer script, sprite nem collider.

Tambem crie CenterFromNorthSpawn, CenterFromEastSpawn, CenterFromSouthSpawn e CenterFromWestSpawn se o Player puder retornar de qualquer portao.

Para uma cena chamada Portao, a ancora exigida e GateEntranceSpawn. O projeto so reconhece Centro e Portao; uma terceira cena e tratada como centro pelo codigo atual.

A cena deve ter exatamente um PlayerInteractor ativo; o Player deve ter Rigidbody2D para a sincronizacao fisica de posicao. Inclua a cena em Build Settings quando ela for carregada.

Relacao geral

GameSession centraliza GameSaveData e cria os servicos. GameSaveRepository persiste no PlayerPrefs. Economia, cidade, inventario, campanha e navegacao compartilham esse estado.

PlayerInteractor detecta IInteractable; NPCs e predios abrem interfaces ou chamam servicos. UiMenuState e locks modais bloqueiam movimentacao durante interfaces.

CityTravelNpc abre TravelConversationPanel; SceneTransitionController salva destino, faz fade, carrega a cena; PlayerSpawnController escolhe a ancora e posiciona o Player.

CampaignRuntimeDriver avanca a campanha; WaveDirectorService agenda ondas; BattleResolutionService resolve ataques; ThreatPresentationService e HUDs apresentam os avisos.

Core, persistencia e entrada

Core/UnitType.cs - enum de Archer, Warrior e Mage; base para exercito, loja e campanha.

Core/GameSession.cs - composicao central dos servicos e singleton GameSessionProvider.

Core/PlayerInventoryService.cs - recruta aventureiros e armazena itens, usando EconomyService e save.

Core/Time/IClock.cs - interface de relogio e SystemClock; usada por estoque e impostos.

Core/Persistence/GameSaveData.cs - DTO do dinheiro, predios, impostos, tropas, inventario, campanha e navegacao.

Core/Persistence/GameSaveRepository.cs - le/grava PlayerPrefs, migra save antigo, normaliza e recupera save corrompido.

Core/Persistence/CampaignSaveData.cs - dados e normalizacao de portoes, ondas, relatorios e transferencias.

Core/Persistence/NavigationSaveData.cs - area atual, portao ativo e porta de retorno ao centro.

Core/Input/GameInput.cs - acoes do Input System: mover, interagir, confirmar e cancelar.

Core/Input/IInteractable.cs - contratos para interacao, prompt, saida de alcance e modal.

Core/Input/PlayerInteractor.cs - seleciona o interagivel mais proximo e administra locks do Player.

Core/Input/UiMenuState.cs - registra menus abertos para coordenar UI e prompt.

Core/Input/InteractionPromptPresenter.cs - HUD persistente de prompt, carregado de Resources/UI/InteractionPrompt.

Player/moviment.cs - move Rigidbody2D, respeita bloqueio modal e limita Player a camera.

Economia, cidade e loja

Economy/EconomyService.cs - fonte de verdade do ouro, persiste e emite MoneyChanged.

City/BuildingTypes.cs - MagicSchool, ArcherStand, Barracks e LocalMarket.

City/BuildingDefinition.cs - ScriptableObject com dados de predio.

City/BuildingCalculator.cs - custo de melhoria e producao por nivel.

City/BuildingService.cs - registra definicoes, compra upgrades e publica LevelChanged.

City/BuildingBehaviour.cs - predio interagivel da cena; atualiza a exibicao de nivel.

City/CityManager.cs - conecta predios, painel, economia e BuildingService na cena.

City/TaxCalculator.cs - calculos puros de imposto e limite de acumulacao.

City/TaxAccrualService.cs - acumula imposto do Mercado Local e o salva em pausa/destruicao.

City/TaxCollectorNpc.cs - coleta impostos e mostra resultado no AlertPopup.

Portao/RecruitmentService.cs - estoque, preco, reposicao em 6h e compra de tropas; capacidade depende dos predios.

Portao/RecruitmentRowView.cs - linha de compra que valida quantidade e publica eventos.

Portao/NPCVenda.cs - NPC da loja; abre modal, atualiza estoque/tempo e usa AlertPopup.

Aventureiros e Canvas

Aventureiro/Item.cs - ScriptableObject de item, ID persistente e bonus.

Aventureiro/Tracos.cs - ScriptableObject marcador de tracos.

Aventureiro/Char_base.cs - definicao de aventureiro: atributos, classe, preco, itens e escalas.

Aventureiro/AdventurerStats.cs - calcula vida/dano; RuntimeState mantem vida corrente.

Aventureiro/Aventureiro.cs - NPC recrutavel que solicita abertura ao controller do Player.

Player/AdventurerRecruitmentController.cs - seleciona, mostra painel e compra via PlayerInventoryService.

Canvas/AdventurerRecruitmentPanel.cs - apresenta dados e emite pedido de compra.

Canvas/GoldBalancePresenter.cs - atualiza texto do ouro ao receber MoneyChanged.

Canvas/BuildingPanel.cs - mostra dados de predio, pede upgrade e bloqueia movimento.

Canvas/AlertPopup.cs - mensagem temporaria reutilizavel.

Navegacao

Navigation/NavigationTypes.cs - tipos de area/destino, snapshots e nomes dos portoes.

Navigation/SceneCatalog.cs - nomes das cenas Centro e Portao.

Navigation/NavigationService.cs - entra no portao, retorna ao centro e persiste o contexto.

Navigation/CityTravelNpc.cs - NPC de destino configuravel que abre conversa.

Navigation/TravelConversationPanel.cs - UI persistente de confirmacao/cancelamento de viagem.

Navigation/SceneTransitionController.cs - fade, checkpoint, LoadSceneAsync e reposicionamento.

Navigation/PlayerSpawnController.cs - escolhe CenterDefaultSpawn, CenterFrom[Portao]Spawn ou GateEntranceSpawn e reposiciona Player/Rigidbody2D.

Navigation/GatePresentationFormatter.cs - formata textos de apresentacao do portao.

Navigation/GateContextPresenter.cs - HUD do portao e ameaca, carregado de Resources/UI/GateContext.

Campanha e ferramentas

Campaign/CampaignTypes.cs - snapshots de portao, onda, inimigo, transferencia e batalha.

Campaign/CampaignRules.cs - regras de poder, custo, progressao e efetividade.

Campaign/CampaignRandom.cs - aleatoriedade persistivel e reprodutivel.

Campaign/WaveGenerator.cs - gera atraso, alvo, poder e composicao de ondas.

Campaign/WaveDirectorService.cs - agenda ondas, avisos e resolucoes.

Campaign/CampaignService.cs - relogio, pausa, checkpoint e fachada da campanha.

Campaign/TroopDeploymentService.cs - despacha tropas entre reserva e portoes.

Campaign/BattleCalculator.cs - calcula confronto de defesa e inimigos.

Campaign/BattleResolutionService.cs - resolve ataques, altera portoes e cria relatorios.

Campaign/ThreatForecastService.cs - estima defesa, ameaca e vantagem.

Campaign/ThreatPresentationService.cs - converte estado em dados/textos de UI.

Campaign/CampaignRuntimeDriver.cs - singleton que atualiza campanha por frame e salva.

Campaign/GlobalThreatHud.cs - HUD persistente de alertas de ondas/batalhas.

Campaign/GarrisonManagementPanel.cs - UI para enviar/receber tropas e ler relatorios.

Campaign/CaptainNpc.cs - NPC que abre a guarnicao e indica ameacas.

Campaign/WaveAlertPresentationConfig.cs - configuracao visual de alertas.

Editor/CityNavigationSceneSetup.cs - menu que cria marcadores/NPCs nas cenas Centro e Portao e ajusta Build Settings.

Editor/UiPrefabSetup.cs - gera prefabs de prompt, fade, viagem, guarnicao e alertas.